

Week 8 - Graded Mini Project

Learning Outcome Addressed

- Data Extraction and Manipulation Using SQL and Pandas
- Deriving Business Insights through Exploratory Data Analysis and Visualisation

Objective

This mini project is designed to help you explore and analyse retail sales data using SQL, Pandas, and Python visualisations. You'll work with two datasets: one containing stock information and another recording sales transactions. Through a combination of structured queries and Python-based EDA, you'll derive business insights such as bestselling items, customer purchasing patterns, and revenue trends.

Submission Instructions

Please document your response on the following pages.

Once you have completed the activity, save the file as a PDF and upload it. Be sure to name the file as **Module 8: Graded Mini Project_[Your last name]**.

Your submission will be considered complete when it meets the following criteria:

- Includes all the key elements outlined in the activity instructions and the rubric.
- Add screenshots of visual outputs wherever required.
- Adheres to the submission guidelines.
- Is submitted on time.

This is a required activity and counts towards programme completion.

Reflect on the task and respond to the following questions.

A. Basic SQL Queries (5 Tasks)- Load csv files in database. (Note: Kindly provide the solution in the form of an SQL query.)

Question: Retrieve all stock items that contain the word "**T-LIGHT**" in their description.

```
select *  
  
from stockdetails  
  
where UPPER(`Description`) like '%T-LIGHT%';
```

Question: Calculate the **total quantity sold per StockCode**.

```
select StockCode, SUM(Quantity) as total_quantity  
  
from Sales  
  
group by StockCode  
  
order by total_quantity desc;
```

Question: Find **total revenue (Quantity * UnitPrice)** per **CustomerID**.

```
select CustomerID, SUM(Quantity * UnitPrice) as total_revenue  
  
from Sales  
  
group by CustomerID  
  
order by total_revenue desc;
```

Question: Get a list of all **invoices** and count of **distinct stock items** per invoice.

```
select InvoiceNo, COUNT(DISTINCT StockCode) as distinct_items  
  
from sales  
  
group by InvoiceNo  
  
order by distinct_items desc;
```

Question: Perform an **inner join** between sales and stock details to display full item names along with **total revenue per item**.

```
select a.StockCode, b.Description as item_name, SUM(a.Quantity * a.UnitPrice) as  
total_revenue  
  
from Sales a  
  
INNER JOIN StockDetails b
```

```
on a.StockCode = b.StockCode

group by a.StockCode, b.Description

order by total_revenue desc;
```

B. EDA Using Pandas (5 Tasks) (Note: Kindly provide the lines of python code.)

Question: Check for **missing values**, **data types**, and **duplicates** in both datasets.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

print("Sales missing:\n", sales.isna().sum())

print("Stock missing:\n", stock.isna().sum())

print("Sales data types:\n", sales.dtypes)

print("Stock data types:\n", stock.dtypes)

print("Sales duplicate rows:", sales.duplicated().sum())

print("Stock duplicate rows:", stock.duplicated().sum())
```

Question: Convert InvoiceDate into datetime, and extract:

- Invoice date
- Month
- Hour of transaction

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

sales['InvoiceDate'] = pd.to_datetime(sales['InvoiceDate'], errors='coerce')
```

```
sales['InvoiceDateOnly'] = sales['InvoiceDate'].dt.date

sales['Month'] = sales['InvoiceDate'].dt.to_period('M').astype(str)

sales['Hour'] = sales['InvoiceDate'].dt.hour
```

Question: Add a new column TotalPrice = Quantity × UnitPrice.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

sales['TotalPrice'] = sales['Quantity'] * sales['UnitPrice']
```

Question: Identify the **top 3 bestselling items** by quantity sold.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

top3_qty =
(sales.groupby('StockCode')['Quantity'].sum().sort_values(ascending=False).head(3))

print(top3_qty)
```

Question: Find out how many **unique customers** made purchases and the **average quantity per invoice**.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')
```

```
unique_customers = sales['CustomerID'].nunique()

avg_qty_per_invoice = sales.groupby('InvoiceNo')['Quantity'].sum().mean()

print("Unique customers:", unique_customers)

print("Average quantity per invoice:", avg_qty_per_invoice)
```

C. Visualisation (5 Tasks) (Note: Kindly provide the lines of python code and the screenshots of charts and plots.)

Question: Plot a **bar chart** of top 10 items by **quantity sold**.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

import matplotlib.pyplot as plt

qty_per_item =
sales.groupby('StockCode')['Quantity'].sum().sort_values(ascending=False).head(10)

plt.figure()

qty_per_item.plot(kind='bar')

plt.xlabel('StockCode'); plt.ylabel('Total Quantity Sold'); plt.title('Top 10 Items by Quantity Sold')

plt.tight_layout(); plt.savefig('top10_items_by_quantity.png'); plt.close()
```

Question: Plot **total sales per hour of the day** to understand peak shopping hours.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')
```

```
import matplotlib.pyplot as plt

sales_per_hour = sales.groupby('Hour')['TotalPrice'].sum().reindex(range(24), fill_value=0)

plt.figure()

sales_per_hour.plot(kind='line', marker='o')

plt.xlabel('Hour of Day'); plt.ylabel('Total Sales'); plt.title('Total Sales per Hour')

plt.tight_layout(); plt.savefig('sales_per_hour.png'); plt.close()
```

Question: Create a **pie chart** showing revenue distribution among top 5 customers.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

import matplotlib.pyplot as plt

rev_by_cust =
(sales.groupby('CustomerID')['TotalPrice'].sum().sort_values(ascending=False).head(5))

plt.figure()

rev_by_cust.plot(kind='pie', autopct='%1.1f%%')

plt.ylabel(""); plt.title('Revenue Distribution among Top 5 Customers')

plt.tight_layout(); plt.savefig('top5_customers_revenue_pie.png'); plt.close()
```

Question: Visualise the **monthly revenue trend** using a line chart.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

import matplotlib.pyplot as plt
```

```
monthly_rev = sales.groupby('Month')['TotalPrice'].sum().sort_index()

plt.figure()

monthly_rev.plot(kind='line', marker='o')

plt.xlabel('Month'); plt.ylabel('Revenue'); plt.title('Monthly Revenue Trend')

plt.xticks(rotation=45, ha='right')

plt.tight_layout(); plt.savefig('monthly_revenue_trend.png'); plt.close()
```

Question: Create a **stacked bar chart** of top 5 invoices by revenue, showing contribution from each item.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

import matplotlib.pyplot as plt

invoice_rev_idx =
sales.groupby('InvoiceNo')['TotalPrice'].sum().sort_values(ascending=False).head(5).index

pivot = (sales[sales['InvoiceNo'].isin(invoice_rev_idx)]

        .pivot_table(index='InvoiceNo', columns='StockCode', values='TotalPrice', aggfunc='sum')

        .fillna(0))

pivot_top_cols = pivot.loc[:, pivot.sum().sort_values(ascending=False).head(10).index]

plt.figure()

pivot_top_cols.plot(kind='bar', stacked=True)

plt.xlabel('InvoiceNo'); plt.ylabel('Revenue'); plt.title('Top 5 Invoices by Revenue (Stacked by Item)')

plt.legend(title='StockCode', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout(); plt.savefig('stacked_top5_invoices.png'); plt.close()
```

D. Business Insights (5 Tasks) (Note: Kindly provide the lines of python code.)

Question: Identify which **product generates the highest total revenue**.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

product_revenue =
sales.groupby('StockCode')['TotalPrice'].sum().sort_values(ascending=False)

top_product = product_revenue.head(1)

print(top_product)
```

Question: Determine **average order value per invoice**.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

aov = sales.groupby('InvoiceNo')['TotalPrice'].sum().mean()

print("Average Order Value:", aov)
```

Question: Find the **customer with the highest number of transactions**.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

cust_txn_counts =
sales.groupby('CustomerID')['InvoiceNo'].nunique().sort_values(ascending=False)

top_customer_by_txn = cust_txn_counts.head(1)

print(top_customer_by_txn)
```

Question: Check how many products in the sales data do **not have a matching entry** in the stockDetails file.

```
import pandas as pd

import numpy as np

sales = pd.read_csv('sales.csv')

stock = pd.read_csv('stockdetails.csv')

sales_codes = set(sales['StockCode'].unique())

stock_codes = set(stock['StockCode'].unique())

missing_codes = sales_codes - stock_codes

print("Missing product count:", len(missing_codes))

missing_list = sorted(list(missing_codes))
```